

Let us assume that you are given the ontology which describes, in a structured form, the entity types and the relationships among them. Now you are also given the text corpus which contains all the documents of interest. This could be the entire World Wide Web (WWW) or a small complete set of documents, such as an enterprise document store. For our example, let us consider that you are given a medical ontology of the type we had discussed earlier. The document corpus can be the entire WWW. Given that we know the entity types and relation types in the ontology, we first need to identify the entity instances of the relevant types. For our example, we need to find DISEASE entities, DRUG entities and SYMPTOM entities, which are mentioned in the corpus. Standard NLP techniques such as Named Entity Recognition can be modified to identify entities, and then the identified entity mentions are analysed to determine which entity type they belong to.

Entity typing can be done by means of: (a) the supervised technique, (b) the weakly-supervised technique, and (c) the distantly supervised technique. Standard supervised techniques use annotated data, and entity typing is a multi-label classification problem where each entity is classified into the right entity type label, using labelled data to learn the prediction function. Weakly supervised techniques use pattern based approaches to identify the entity types. Typically, a few seed instances and seed patterns are provided to bootstrap the extraction process. For instance, a sentence such as "Metformin is the first line drug of choice in the treatment of diabetes," is marked with 'metformin' as the seed drug instance, and the seed pattern extracted is 'drug ... treatment of ...'. This pattern can then be used as an extractor pattern on the remaining corpus to extract other similar drug disease mentions.

Weakly supervised techniques require that seed instances should be specified for each of the different entity and relation types that need to be extracted. Distantly supervised techniques do not need explicit seed instances and seed patterns. Instead, they use available knowledge bases such as Freebase and Wikipedia to obtain the seed instances automatically.

While entity typing is a hard problem, there is another associated challenge linked to it, which is not often discussed. Consider our problem of extracting diseases. Let us assume that we have two sentences, such as the following:

1. Diabetes has multiple causes, the primary one being that blood sugar levels are not being controlled adequately.
2. Diabetes mellitus is a serious disease affecting millions and Metformin is the first line drug of choice against it.

Assume that we have already identified the entity mentions 'diabetes' and 'diabetes mellitus' from the above two sentences. We have also been able to determine that 'diabetes' and 'diabetes mellitus' are of entity type 'DISEASE' and 'Metformin' is of entity type 'DRUG'. Now the main question that confronts us is the following: Are the two entity mentions 'diabetes' and 'diabetes mellitus' referring to the same entity instance or different ones? If they are referring to the same entity instance, what should be the knowledge base entity reference which will be used to refer to both these entities?

This is the problem of canonicalisation of knowledge bases, wherein the entity mentions in the unstructured text need to be mapped to a unique structured database entity.

Before we discuss how we can address knowledge base canonicalisation, you may be wondering why these two entity mentions need to be canonicalised at all? Why can't we create two rows in the knowledge base table, one for diabetes and the second for diabetes mellitus such that the facts present in sentences (1) and (2) above are included in those two rows independently? Well, the answer is that canonicalisation of knowledge bases allows us to unify the facts we have extracted so that we can present a more complete answer to a query. For instance, if there is a query such as, "What is the primary cause of diabetes mellitus?" to our knowledge base, we need to provide the answer, "Inadequate blood sugar control." However, unless we unify the two entity mentions of 'diabetes' and 'diabetes mellitus' to a single database instance ID, we can't combine the two facts and provide the right answer. Therefore, knowledge base canonicalisation is needed to avoid storing ambiguous and redundant facts, and to provide a complete answer to user queries.

As you may have realised, one of the simplest approaches for canonicalising these two entity mentions is by means of clustering similar words and phrases. Clustering using word similarity would result in 'diabetes' and 'diabetes mellitus' being assigned the same cluster and, hence, by assigning a unique cluster representative instance which will represent all the cluster members, canonicalisation can be achieved. But this does not always work. For instance, consider the two entity mentions: Type 1 diabetes and Type 2 diabetes. Now, clustering based on similarity would end up clustering these two instances and assigning a single unique KB entity ID. However, these are two distinct diseases and have different drugs and treatment plans. By forcing these two entities to be clustered into the same knowledge base instance ID, we actually lose precision in providing an answer to a user query of the form: "What drugs are used to treat Type 1 diabetes?" Here is a takeaway question to our readers. In the context of the over-clustering issue, how will you come up with an efficient technique for knowledge base canonicalisation of entity mentions? We will discuss this further in our next column.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

By: Sandya Mannarsamy

The author is an expert in systems software and is currently working as a research scientist at Xerox India Research Centre. Her interests include compilers, programming languages, file systems and natural language processing. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group *Computer Science Interview Training India* at <http://www.linkedin.com/groups?home=HYPERLINK> <http://www.linkedin.com/groups?home=&gid=2339182> &HYPERLINK <http://www.linkedin.com/groups?home=&gid=2339182>